



Documentación completa del proyecto

Aplicación móvil de aprendizaje de idiomas mediante flashcards y repetición espaciada

Institución: *Universidad Politécnica de Aguascalientes*

Materia: Inteligencia de negocios

Integrantes:

- Alondra Jazmin Tapia Cruz
- Santiago Alonso Silva Pedroza
- Carlos Ríos Yáñez
- José Javier Vizcaino Marquez
- Diego Fernández Martínez
- Jessica Ximena Medina Gutiérrez

Encargado de base de datos: *Alondra y Jessica*

Fecha: *28 de septiembre de 2026*

1. Descripción general

Kian es una aplicación móvil de aprendizaje de idiomas mediante flashcards (tarjetas visuales) con un sistema de repetición espaciada. El usuario ve una imagen, la voltea para ver la traducción correspondiente y califica qué tan bien la recordó. El sistema determina cuándo debe volver a mostrarse cada tarjeta según la respuesta.

Componente	Tecnología
Frontend	React Native + TypeScript (Expo)
Backend	ASP.NET Core Web API + C#
Base de datos	Azure SQL Database
Almacenamiento de imágenes	Azure Blob Storage
ORM	Entity Framework Core
Modo actual	Un solo usuario (sin autenticación; se añadirá en fase futura)

1.1 Objetivo general

Desarrollar una aplicación móvil que permita aprender vocabulario de diferentes idiomas mediante imágenes, traducciones y un sistema de repetición espaciada.

1.2 Objetivos específicos

- Crear una aplicación móvil multiplataforma utilizando React Native.
- Desarrollar una API REST utilizando ASP.NET Core.
- Implementar una base de datos relacional utilizando Azure SQL Database.
- Utilizar Azure Blob Storage para almacenar las imágenes.
- Relacionar cada imagen con una o varias traducciones.
- Permitir seleccionar el idioma de aprendizaje.
- Mostrar las tarjetas mediante una interfaz de flashcards.
- Registrar las respuestas del usuario durante las sesiones de estudio.
- Implementar un sistema básico de repetición espaciada.
- Mantener separado el almacenamiento de imágenes y los datos estructurados.
- Diseñar la arquitectura de manera que pueda crecer en futuras versiones.

2. Arquitectura del sistema

React Native (kianMobile) --> HTTPS/REST --> ASP.NET Core (KIAN) --> Azure SQL Database

|
v

El frontend nunca accede directo a la base de datos ni a Blob Storage. El backend concentra toda la lógica: valida datos, consulta Azure SQL, sube y lee imágenes, y genera URLs temporales de solo lectura. Azure SQL guarda datos estructurados; Blob Storage guarda únicamente los archivos de imagen.

2.1 Responsabilidad de cada componente

React Native: interfaz de usuario, navegación, flashcards, selección de idioma y comunicación con la API. No tiene acceso directo a la base de datos.

ASP.NET Core: recibe solicitudes HTTP, valida información, consulta Azure SQL, sube/lee imágenes de Blob Storage, genera URLs temporales, crea tarjetas, obtiene tarjetas pendientes, registra revisiones y calcula el siguiente intervalo.

Azure SQL Database: almacena usuarios, tarjetas, traducciones y progreso.

Azure Blob Storage: almacena los archivos de imagen. La base de datos guarda únicamente la referencia al blob.

3. Estructura de carpetas

Carpeta raíz del repositorio: *[NOMBRE DEL REPOSITORIO / CARPETA RAÍZ]*

```
[NOMBRE DEL REPOSITORIO]/      (carpeta raíz - repositorio Git)
├── .gitignore
├── KIAN/                        (backend ASP.NET Core)
│   ├── Controllers/
│   │   ├── CardsController.cs
│   │   └── ReviewsController.cs
│   ├── Models/
│   │   ├── User.cs
│   │   ├── Card.cs
│   │   ├── CardTranslation.cs
│   │   └── CardReview.cs
│   ├── DTOs/
│   │   ├── CreateCardDto.cs
│   │   ├── AddTranslationDto.cs
│   │   ├── CardResponseDto.cs
│   │   └── ReviewRequestDto.cs
│   ├── Data/
│   │   └── AppDbContext.cs
│   ├── Services/
│   │   ├── CurrentUserService.cs
│   │   ├── BlobStorageService.cs
│   │   ├── SpacedRepetitionService.cs
│   │   └── CardService.cs
│   ├── Properties/
│   │   └── launchSettings.json
│   ├── appsettings.json
│   ├── KIAN.http
│   ├── KIAN.csproj
│   └── Program.cs
├── kianMobile/                (frontend React Native / Expo)
│   ├── src/
│   │   ├── components/
│   │   │   ├── Flashcard.tsx
│   │   │   ├── Loading.tsx
│   │   │   └── ReviewButtons.tsx
│   │   ├── screens/
│   │   │   ├── HomeScreen.tsx
│   │   │   ├── StudyScreen.tsx
│   │   │   └── LanguagesScreen.tsx
│   │   ├── services/
│   │   │   └── api.ts
│   │   ├── types/
│   │   │   └── Card.ts
│   │   ├── navigation/
│   │   │   └── AppNavigation.tsx
│   │   ├── context/
│   │   │   └── SessionContext.tsx
│   ├── App.tsx
│   ├── package.json
│   └── tsconfig.json
```

4. Base de datos (Azure SQL)

Diseñada por *[NOMBRE DEL ENCARGADO DE BASE DE DATOS]*. El sistema opera en modo de un solo usuario, pero conserva la tabla Users porque las tablas Cards y CardReviews la referencian.

4.1 Script de creación de tablas

```
CREATE TABLE Users (
    Id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    Name NVARCHAR(100) NOT NULL,
    Email NVARCHAR(255) NOT NULL UNIQUE,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE TABLE Cards (
    Id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    UserId UNIQUEIDENTIFIER NOT NULL,
    ImageBlobName NVARCHAR(500) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT FK_Cards_Users FOREIGN KEY (UserId) REFERENCES Users(Id)
);

CREATE TABLE CardTranslations (
    Id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    CardId UNIQUEIDENTIFIER NOT NULL,
    LanguageCode NVARCHAR(10) NOT NULL,
    TranslatedText NVARCHAR(500) NOT NULL,
    ExampleSentence NVARCHAR(1000) NULL,
    CONSTRAINT FK_Translations_Cards FOREIGN KEY (CardId) REFERENCES Cards(Id),
    CONSTRAINT UQ_Card_Language UNIQUE (CardId, LanguageCode)
);

CREATE TABLE CardReviews (
    Id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    CardId UNIQUEIDENTIFIER NOT NULL,
    UserId UNIQUEIDENTIFIER NOT NULL,
    NextReviewAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    IntervalDays INT NOT NULL DEFAULT 0,
    EaseFactor DECIMAL(5,2) NOT NULL DEFAULT 2.50,
    Repetitions INT NOT NULL DEFAULT 0,
    Lapses INT NOT NULL DEFAULT 0,
    CONSTRAINT FK_Reviews_Cards FOREIGN KEY (CardId) REFERENCES Cards(Id),
    CONSTRAINT FK_Reviews_Users FOREIGN KEY (UserId) REFERENCES Users(Id),
    CONSTRAINT UQ_Review_User_Card UNIQUE (UserId, CardId)
);
```

4.2 Relaciones principales

- User → Cards → CardTranslations
- User → CardReviews
- Cards → CardReviews

Una tarjeta puede tener múltiples traducciones (una por idioma como máximo). Una tarjeta tiene un registro de progreso por usuario.

4.3 Datos de conexión

Dato	Valor
Servidor	[NOMBRE-DEL-SERVIDOR].database.windows.net
Base de datos	[NOMBRE DE LA BASE DE DATOS]
Usuario administrador	[USUARIO ADMINISTRADOR]
Contraseña	[CONTRASEÑA — guardar solo en User Secrets, nunca en el repositorio]
Cuenta de Storage	[NOMBRE DE LA CUENTA DE STORAGE]
Contenedor de Blob Storage	[NOMBRE DEL CONTENEDOR, ej. flashcard-images]

5. Backend (ASP.NET Core)

5.1 Paquetes NuGet requeridos

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Azure.Storage.Blobs
```

5.2 Models

Models/User.cs

```
namespace KIAN.Models;

public class User
{
    public Guid Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public List<Card> Cards { get; set; } = new();
    public List<CardReview> Reviews { get; set; } = new();
}
```

Models/Card.cs

```
namespace KIAN.Models;

public class Card
{
    public Guid Id { get; set; }
    public Guid UserId { get; set; }
    public string ImageBlobName { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public User? User { get; set; }
    public List<CardTranslation> Translations { get; set; } = new();
    public List<CardReview> Reviews { get; set; } = new();
}
```

Models/CardTranslation.cs

```
namespace KIAN.Models;

public class CardTranslation
{
    public Guid Id { get; set; }
    public Guid CardId { get; set; }
    public string LanguageCode { get; set; } = string.Empty;
    public string TranslatedText { get; set; } = string.Empty;
    public string? ExampleSentence { get; set; }

    public Card? Card { get; set; }
}
```

Models/CardReview.cs

```
namespace KIAN.Models;

public class CardReview
{
    public Guid Id { get; set; }
    public Guid CardId { get; set; }
    public Guid UserId { get; set; }
    public DateTime NextReviewAt { get; set; } = DateTime.UtcNow;
    public int IntervalDays { get; set; } = 0;
    public decimal EaseFactor { get; set; } = 2.50m;
    public int Repetitions { get; set; } = 0;
    public int Lapses { get; set; } = 0;

    public Card? Card { get; set; }
    public User? User { get; set; }
}
```

5.3 DTOs

DTOs/CreateCardDto.cs

```
using System.ComponentModel.DataAnnotations;

namespace KIAN.DTOs;

public class CreateCardDto
{
    [Required]
    public IFormFile Image { get; set; } = default!;

    [Required, StringLength(10)]
    public string LanguageCode { get; set; } = string.Empty;

    [Required, StringLength(500)]
    public string TranslatedText { get; set; } = string.Empty;

    [StringLength(1000)]
    public string? ExampleSentence { get; set; }
}
```

DTOs/AddTranslationDto.cs

```
using System.ComponentModel.DataAnnotations;

namespace KIAN.DTOs;

public class AddTranslationDto
{
    [Required, StringLength(10)]
    public string LanguageCode { get; set; } = string.Empty;

    [Required, StringLength(500)]
    public string TranslatedText { get; set; } = string.Empty;

    [StringLength(1000)]
    public string? ExampleSentence { get; set; }
}
```

DTOs/CardResponseDto.cs

```
namespace KIAN.DTOs;

public class TranslationDto
{
    public string LanguageCode { get; set; } = string.Empty;
    public string TranslatedText { get; set; } = string.Empty;
    public string? ExampleSentence { get; set; }
}

public class CardResponseDto
{
    public Guid Id { get; set; }
}
```

```
public string imageUrl { get; set; } = string.Empty;
public DateTime CreatedAt { get; set; }
public List<TranslationDto> Translations { get; set; } = new();
}
```

DTOs/ReviewRequestDto.cs

```
using System.ComponentModel.DataAnnotations;

namespace KIAN.DTOs;

public class ReviewRequestDto
{
    [Required]
    public Guid CardId { get; set; }

    // 0 = Otra vez, 1 = Dificil, 2 = Bien, 3 = Facil
    [Range(0, 3)]
    public int Quality { get; set; }
}
```

5.4 Data — AppDbContext

Data/AppDbContext.cs

```
using KIAN.Models;
using Microsoft.EntityFrameworkCore;

namespace KIAN.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<User> Users => Set<User>();
    public DbSet<Card> Cards => Set<Card>();
    public DbSet<CardTranslation> CardTranslations => Set<CardTranslation>();
    public DbSet<CardReview> CardReviews => Set<CardReview>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        b.Entity<User>(e =>
        {
            e.ToTable("Users");
            e.Property(x => x.Name).HasMaxLength(100).IsRequired();
            e.Property(x => x.Email).HasMaxLength(255).IsRequired();
            e.Property(x => x.CreatedAt).HasColumnType("datetime2");
            e.HasIndex(x => x.Email).IsUnique();
        });

        b.Entity<Card>(e =>
        {
            e.ToTable("Cards");
            e.Property(x => x.ImageBlobName).HasMaxLength(500).IsRequired();
            e.Property(x => x.CreatedAt).HasColumnType("datetime2");
            e.HasIndex(x => x.UserId);
            e.HasOne(x => x.User)
                .WithMany(u => u.Cards)
                .HasForeignKey(x => x.UserId)
                .OnDelete(DeleteBehavior.Restrict);
        });

        b.Entity<CardTranslation>(e =>
        {
            e.ToTable("CardTranslations");
            e.Property(x => x.LanguageCode).HasMaxLength(10).IsRequired();
            e.Property(x => x.TranslatedText).HasMaxLength(500).IsRequired();
            e.Property(x => x.ExampleSentence).HasMaxLength(1000);
            e.HasIndex(x => x.LanguageCode);
            e.HasIndex(x => new { x.CardId, x.LanguageCode }).IsUnique();
            e.HasOne(x => x.Card)
                .WithMany(c => c.Translations)
                .HasForeignKey(x => x.CardId)
                .OnDelete(DeleteBehavior.Restrict);
        });

        b.Entity<CardReview>(e =>
        {
            e.ToTable("CardReviews");
```

```
e.Property(x => x.NextReviewAt).HasColumnType("datetime2");
e.Property(x => x.EaseFactor).HasColumnType("decimal(5,2)");
e.HasIndex(x => x.NextReviewAt);
e.HasIndex(x => new { x.UserId, x.CardId }).IsUnique();

e.HasOne(x => x.Card)
  .WithMany(c => c.Reviews)
  .HasForeignKey(x => x.CardId)
  .OnDelete(DeleteBehavior.Restrict);

e.HasOne(x => x.User)
  .WithMany(u => u.Reviews)
  .HasForeignKey(x => x.UserId)
  .OnDelete(DeleteBehavior.Restrict);
});
}
```

5.5 Services

Services/CurrentUserService.cs

```
namespace KIAN.Services;

// Modo un solo usuario: guarda el Id del usuario unico de la aplicacion.
// Se inicializa al arrancar la API (ver Program.cs).
public class CurrentUserService
{
    private Guid? _userId;

    public Guid UserId => _userId
        ?? throw new InvalidOperationException("El usuario por defecto no se ha inicializado.");

    public void Set(Guid id) => _userId = id;
}
```

Services/BlobStorageService.cs

```
using Azure.Storage.Blobs;
using Azure.Storage.Blobs.Models;
using Azure.Storage.Sas;

namespace KIAN.Services;

public class BlobStorageService
{
    private readonly BlobContainerClient _container;

    public BlobStorageService(IConfiguration config)
    {
        var connectionString = config["AzureStorage:ConnectionString"]
            ?? throw new InvalidOperationException("Falta AzureStorage:ConnectionString");
        var containerName = config["AzureStorage:ContainerName"] ?? "flashcard-images";

        _container = new BlobContainerClient(connectionString, containerName);
    }

    public async Task<string> UploadAsync(IFormFile file)
    {
        await _container.CreateIfNotExistsAsync(PublicAccessType.None);

        var extension = Path.GetExtension(file.FileName).ToLowerInvariant();
        var blobName = $"cards/{Guid.NewGuid():N}{extension}";

        var blob = _container.GetBlobClient(blobName);
        await using var stream = file.OpenReadStream();
        await blob.UploadAsync(stream, new BlobHttpHeaders { ContentType = file.ContentType });
    }

    return blobName;
}

// URL temporal de solo lectura (las imagenes permanecen privadas)
```

```
public string GetTemporaryUrl(string blobName, int minutes = 30)
{
    var blob = _container.GetBlobClient(blobName);

    if (!blob.CanGenerateSasUri)
        return string.Empty;

    return blob.GenerateSasUri(BlobSasPermissions.Read,
        DateTimeOffset.UtcNow.AddMinutes(minutes)).ToString();
}

public async Task DeleteAsync(string blobName)
{
    await _container.GetBlobClient(blobName).DeleteIfExistsAsync();
}
}
```

Services/SpacedRepetitionService.cs

```
using KIAN.Models;

namespace KIAN.Services;

// Version simplificada de SM-2.
// Calidad: 0 = Otra vez, 1 = Dificil, 2 = Bien, 3 = Facil
public class SpacedRepetitionService
{
    private const decimal MinEase = 1.30m;
    private const decimal MaxEase = 5.00m;

    public void Apply(CardReview review, int quality)
    {
        var now = DateTime.UtcNow;

        switch (quality)
        {
            case 0: // Otra vez
                review.Lapses++;
                review.Repetitions = 0;
                review.IntervalDays = 0;
                review.EaseFactor = Clamp(review.EaseFactor - 0.20m);
                review.NextReviewAt = now.AddMinutes(10);
                return;

            case 1: // Dificil
                review.EaseFactor = Clamp(review.EaseFactor - 0.15m);
                review.IntervalDays = Math.Max(1, (int)Math.Round(review.IntervalDays *
1.2));
                break;

            case 2: // Bien
                review.IntervalDays = review.Repetitions switch
                {
                    0 => 1,
                    1 => 3,
                    _ => Math.Max(1, (int)Math.Round(review.IntervalDays *
(double)review.EaseFactor))
                };
                break;

            case 3: // Facil
                review.EaseFactor = Clamp(review.EaseFactor + 0.15m);
                review.IntervalDays = review.Repetitions switch
                {
                    0 => 3,
                    1 => 5,
                    _ => Math.Max(1, (int)Math.Round(review.IntervalDays *
(double)review.EaseFactor * 1.3))
                };
                break;

            default:
                throw new ArgumentOutOfRangeException(nameof(quality), "La calidad debe
estar entre 0 y 3.");
        }
    }
}
```

```
        review.Repetitions++;  
        review.NextReviewAt = now.AddDays(review.IntervalDays);  
    }  
  
    private static decimal Clamp(decimal value) => Math.Min(MaxEase, Math.Max(MinEase,  
value));  
}
```

Services/CardService.cs

```
using KIAN.Data;
using KIAN.DTOs;
using KIAN.Models;
using Microsoft.EntityFrameworkCore;

namespace KIAN.Services;

public class CardService
{
    private readonly AppDbContext _db;
    private readonly BlobStorageService _blob;
    private readonly SpacedRepetitionService _sr;
    private readonly CurrentUserService _currentUser;

    public CardService(
        AppDbContext db,
        BlobStorageService blob,
        SpacedRepetitionService sr,
        CurrentUserService currentUser)
    {
        _db = db;
        _blob = blob;
        _sr = sr;
        _currentUser = currentUser;
    }

    private Guid UserId => _currentUser.UserId;

    public async Task<CardResponseDto> CreateAsync(CreateCardDto dto)
    {
        var blobName = await _blob.UploadAsync(dto.Image);

        var card = new Card
        {
            UserId = UserId,
            ImageBlobName = blobName,
            Translations =
            {
                new CardTranslation
                {
                    LanguageCode = Normalize(dto.LanguageCode),
                    TranslatedText = dto.TranslatedText.Trim(),
                    ExampleSentence = dto.ExampleSentence?.Trim()
                }
            },
            Reviews =
            {
                new CardReview { UserId = UserId, NextReviewAt = DateTime.UtcNow }
            }
        };

        _db.Cards.Add(card);
        await _db.SaveChangesAsync();

        return Map(card, null);
    }
}
```

```

    public async Task<CardResponseDto> AddTranslationAsync(Guid cardId, AddTranslationDto
dto)
    {
        var card = await _db.Cards
            .Include(c => c.Translations)
            .FirstOrDefaultAsync(c => c.Id == cardId && c.UserId == UserId)
            ?? throw new KeyNotFoundException("Tarjeta no encontrada.");

        var lang = Normalize(dto.LanguageCode);

        if (card.Translations.Any(t => t.LanguageCode == lang))
            throw new InvalidOperationException($"La tarjeta ya tiene una traduccion en
'{lang}'.");

        card.Translations.Add(new CardTranslation
        {
            LanguageCode = lang,
            TranslatedText = dto.TranslatedText.Trim(),
            ExampleSentence = dto.ExampleSentence?.Trim()
        });

        await _db.SaveChangesAsync();
        return Map(card, null);
    }

    public async Task<List<CardResponseDto>> GetAllAsync(string? language)
    {
        var query = _db.Cards
            .AsNoTracking()
            .Include(c => c.Translations)
            .Where(c => c.UserId == UserId);

        if (!string.IsNullOrEmpty(language))
        {
            var lang = Normalize(language);
            query = query.Where(c => c.Translations.Any(t => t.LanguageCode == lang));
        }

        var cards = await query.OrderByDescending(c => c.CreatedAt).ToListAsync();
        return cards.Select(c => Map(c, language)).ToList();
    }

    public async Task<CardResponseDto?> GetByIdAsync(Guid id)
    {
        var card = await _db.Cards
            .AsNoTracking()
            .Include(c => c.Translations)
            .FirstOrDefaultAsync(c => c.Id == id && c.UserId == UserId);

        return card is null ? null : Map(card, null);
    }

    public async Task<List<CardResponseDto>> GetDueAsync(string? language, int limit)
    {
        var now = DateTime.UtcNow;

```

```

        var query = _db.CardReviews
            .AsNoTracking()
            .Include(r => r.Card!).ThenInclude(c => c.Translations)
            .Where(r => r.UserId == UserId && r.NextReviewAt <= now);

        if (!string.IsNullOrEmpty(language))
        {
            var lang = Normalize(language);
            query = query.Where(r => r.Card!.Translations.Any(t => t.LanguageCode ==
lang));
        }

        var reviews = await query
            .OrderBy(r => r.NextReviewAt)
            .Take(limit)
            .ToListAsync();

        return reviews.Select(r => Map(r.Card!, language)).ToList();
    }

    public async Task RegisterReviewAsync(ReviewRequestDto dto)
    {
        var review = await _db.CardReviews
            .FirstOrDefaultAsync(r => r.CardId == dto.CardId && r.UserId == UserId)
            ?? throw new KeyNotFoundException("No existe progreso para esa tarjeta.");

        _sr.Apply(review, dto.Quality);
        await _db.SaveChangesAsync();
    }

    private static string Normalize(string value) => value.Trim().ToLowerInvariant();

    private CardResponseDto Map(Card card, string? language)
    {
        var translations = card.Translations.AsEnumerable();

        if (!string.IsNullOrEmpty(language))
        {
            var lang = Normalize(language);
            translations = translations.Where(t => t.LanguageCode == lang);
        }

        return new CardResponseDto
        {
            Id = card.Id,
            CreatedAt = card.CreatedAt,
            ImageUrl = _blob.GetTemporaryUrl(card.ImageBlobName),
            Translations = translations.Select(t => new TranslationDto
            {
                LanguageCode = t.LanguageCode,
                TranslatedText = t.TranslatedText,
                ExampleSentence = t.ExampleSentence
            }).ToList()
        };
    }
}

```


5.6 Controllers

Controllers/CardsController.cs

```
using KIAN.DTOs;
using KIAN.Services;
using Microsoft.AspNetCore.Mvc;

namespace KIAN.Controllers;

[ApiController]
[Route("api/[controller]")]
public class CardsController : ControllerBase
{
    private static readonly string[] AllowedTypes = { "image/jpeg", "image/png",
    "image/webp" };
    private const long MaxImageBytes = 5 * 1024 * 1024; // 5 MB

    private readonly CardService _cards;

    public CardsController(CardService cards) => _cards = cards;

    // GET /api/cards | GET /api/cards?language=en
    [HttpGet]
    public async Task<ActionResult<List<CardResponseDto>>> GetAll([FromQuery] string?
    language)
    => await _cards.GetAllAsync(language);

    // GET /api/cards/{id}
    [HttpGet("{id:guid}")]
    public async Task<ActionResult<CardResponseDto>> GetById(Guid id)
    {
        var card = await _cards.GetByIdAsync(id);
        return card is null ? NotFound() : card;
    }

    // POST /api/cards (multipart/form-data)
    [HttpPost]
    [Consumes("multipart/form-data")]
    public async Task<ActionResult<CardResponseDto>> Create([FromForm] CreateCardDto dto)
    {
        if (dto.Image.Length == 0 || dto.Image.Length > MaxImageBytes)
            return BadRequest("La imagen debe pesar entre 1 byte y 5 MB.");

        if (!AllowedTypes.Contains(dto.Image.ContentType))
            return BadRequest("Solo se permiten imagenes JPG, PNG o WEBP.");

        var card = await _cards.CreateAsync(dto);
        return CreatedAtAction(nameof(GetById), new { id = card.Id }, card);
    }

    // POST /api/cards/{id}/translations
    [HttpPost("{id:guid}/translations")]
    public async Task<ActionResult<CardResponseDto>> AddTranslation(Guid id,
    AddTranslationDto dto)
    {
        try
```

```

        {
            return await _cards.AddTranslationAsync(id, dto);
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(ex.Message);
        }
        catch (InvalidOperationException ex)
        {
            return Conflict(ex.Message);
        }
    }
}

```

Controllers/ReviewsController.cs

```

using KIAN.DTOs;
using KIAN.Services;
using Microsoft.AspNetCore.Mvc;

namespace KIAN.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ReviewsController : ControllerBase
{
    private readonly CardService _cards;

    public ReviewsController(CardService cards) => _cards = cards;

    // GET /api/reviews/due?language=en&limit=20
    [HttpGet("due")]
    public async Task<ActionResult<List<CardResponseDto>>> GetDue(
        [FromQuery] string? language,
        [FromQuery] int limit = 20)
    {
        limit = Math.Clamp(limit, 1, 100);
        return await _cards.GetDueAsync(language, limit);
    }

    // POST /api/reviews
    [HttpPost]
    public async Task<IActionResult> Register(ReviewRequestDto dto)
    {
        try
        {
            await _cards.RegisterReviewAsync(dto);
            return NoContent();
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(ex.Message);
        }
    }
}

```

5.7 Program.cs

```
using KIAN.Data;
using KIAN.Models;
using KIAN.Services;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Controllers
builder.Services.AddControllers();

// Base de datos (Azure SQL)
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Servicios
builder.Services.AddSingleton<CurrentUserService>();
builder.Services.AddSingleton<BlobStorageService>();
builder.Services.AddSingleton<SpacedRepetitionService>();
builder.Services.AddScoped<CardService>();

// CORS (solo desarrollo)
builder.Services.AddCors(options =>
{
    options.AddPolicy("Dev", p => p.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod());
});

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Usuario unico: usa el primero que exista o crea uno
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    var config = app.Configuration;

    var user = await db.Users.OrderBy(u => u.CreatedAt).FirstOrDefaultAsync();
    if (user is null)
    {
        user = new User
        {
            Name = config["DefaultUser:Name"] ?? "Kian",
            Email = config["DefaultUser:Email"] ?? "kian@local"
        };
        db.Users.Add(user);
        await db.SaveChangesAsync();
    }

    app.Services.GetRequiredService<CurrentUserService>().Set(user.Id);
}

if (app.Environment.IsDevelopment())
{
}
```

```
        app.UseSwagger();
        app.UseSwaggerUI();
        app.UseCors("Dev");
    }
    else
    {
        app.UseHttpsRedirection();
    }

    app.MapControllers();

    app.Run();
```

5.8 appsettings.json (sin secretos)

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "AzureStorage": {
    "ContainerName": "flashcard-images"
  },
  "DefaultUser": {
    "Name": "Kian",
    "Email": "kian@local"
  }
}
```

5.9 Secretos (User Secrets) — nunca en el repositorio

```
cd KIAN
dotnet user-secrets init

dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Server=tcp:[NOMBRE-DEL-SERVIDOR].database.windows.net,1433;Initial Catalog=[NOMBRE DE LA BASE];Persist Security Info=False;User ID=[USUARIO];Password=[CONTRASEÑA];MultipleActiveResultSets=False;Encrypt=True;TrustServerCertificate=False;Connection Timeout=30;"

dotnet user-secrets set "AzureStorage:ConnectionString" "DefaultEndpointsProtocol=https;AccountName=[NOMBRE DE LA CUENTA DE STORAGE];AccountKey=[ACCOUNT KEY];EndpointSuffix=core.windows.net"
```

Verificar con:

```
dotnet user-secrets list
```

5.10 Firewall de Azure SQL

Portal de Azure → SQL Server → Networking → Firewall rules → Add your client IPv4 address → Save. Es necesario para que la API pueda conectarse desde el equipo de desarrollo.

6. Frontend (React Native + TypeScript)

6.1 Crear proyecto e instalar dependencias

```
npx create-expo-app@latest kianMobile --template blank-typescript
cd kianMobile

npx expo install @react-native-async-storage/async-storage react-native-screens react-native-safe-area-context
npm install @react-navigation/native @react-navigation/bottom-tabs
```

6.2 Tips

src/types/Card.ts

```
export interface Translation {
  languageCode: string;
  translatedText: string;
  exampleSentence?: string | null;
}

export interface Card {
  id: string; // Guid
  imageUrl: string;
  createdAt: string;
  translations: Translation[];
}

// 0 = Otra vez, 1 = Dificil, 2 = Bien, 3 = Facil
export type ReviewQuality = 0 | 1 | 2 | 3;
```

6.3 Servicio de API

src/services/api.ts

```
import { Platform } from 'react-native';
import { Card, ReviewQuality } from '../types/Card';

// Ajusta el puerto al de tu perfil "http" en Properties/launchSettings.json
// - Emulador Android: 10.0.2.2
// - Simulador iOS: localhost
// - Telefono fisico: la IP LAN de tu PC (ej. http://[IP-DE-TU-PC]:[PUERTO])
const PORT = [PUERTO DEL BACKEND]; // <-- cambia al puerto real de tu backend
const HOST = Platform.OS === 'android' ? '10.0.2.2' : 'localhost';
export const BASE_URL = `http://${HOST}:${PORT}`;

async function request<T>(path: string, options?: RequestInit): Promise<T> {
  const response = await fetch(`${BASE_URL}${path}`, {
    headers: { 'Content-Type': 'application/json' },
    ...options,
  });

  if (!response.ok) {
    const message = await response.text();
  }
}
```

```

    throw new Error(message || `Error ${response.status}`);
  }

  if (response.status === 204) return undefined as T;
  return (await response.json()) as T;
}

export function getDueCards(language: string, limit = 20): Promise<Card[]> {
  return request<Card[]>(
    `/api/reviews/due?language=${encodeURIComponent(language)}&limit=${limit}`
  );
}

export function getCards(language?: string): Promise<Card[]> {
  const query = language ? `?language=${encodeURIComponent(language)}` : '';
  return request<Card[]>(`/api/cards${query}`);
}

export function sendReview(cardId: string, quality: ReviewQuality): Promise<void> {
  return request<void>('/api/reviews', {
    method: 'POST',
    body: JSON.stringify({ cardId, quality }),
  });
}

```

6.4 Contexto de sesión

[src/context/SessionContext.tsx](#)

```
import React, { createContext, useContext, useEffect, useState, ReactNode } from 'react';
import AsyncStorage from '@react-native-async-storage/async-storage';

interface SessionValue {
  language: string;
  loaded: boolean;
  setLanguage: (language: string) => void;
}

const SessionContext = createContext<SessionValue | undefined>(undefined);

const LANG_KEY = 'kian:language';

export function SessionProvider({ children }: { children: ReactNode }) {
  const [language, setLanguageState] = useState('en');
  const [loaded, setLoaded] = useState(false);

  useEffect(() => {
    (async () => {
      try {
        const stored = await AsyncStorage.getItem(LANG_KEY);
        if (stored) setLanguageState(stored);
      } finally {
        setLoaded(true);
      }
    })();
  }, []);

  const setLanguage = (value: string) => {
    setLanguageState(value);
    AsyncStorage.setItem(LANG_KEY, value);
  };

  return (
    <SessionContext.Provider value={{ language, loaded, setLanguage }}>
      {children}
    </SessionContext.Provider>
  );
}

export function useSession() {
  const ctx = useContext(SessionContext);
  if (!ctx) throw new Error('useSession debe usarse dentro de SessionProvider');
  return ctx;
}
```

6.5 Componentes

[src/components>Loading.tsx](#)

```
import React from 'react';
import { ActivityIndicator, StyleSheet, View } from 'react-native';
```

```
export default function Loading() {
  return (
    <View style={styles.container}>
      <ActivityIndicator size="large" color="#4f46e5" />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center', justifyContent: 'center' },
});
```

src/components/Flashcard.tsx

```
import React from 'react';
import { Image, Pressable, StyleSheet, Text, View } from 'react-native';
import { Card } from '../types/Card';

interface Props {
  card: Card;
  flipped: boolean;
  onFlip: () => void;
}

export default function Flashcard({ card, flipped, onFlip }: Props) {
  const translation = card.translations[0];

  return (
    <Pressable style={styles.card} onPress={onFlip}>
      {!flipped ? (
        <View style={styles.face}>
          <Image source={{ uri: card.imageUrl }} style={styles.image} resizeMode="contain"
        />
        <Text style={styles.hint}>Toca para voltar</Text>
      </View>
      ) : (
        <View style={styles.face}>
          <Text style={styles.word}>{translation?.translatedText ?? '-'}</Text>
          {!!translation?.exampleSentence && (
            <Text style={styles.example}>{translation.exampleSentence}</Text>
          )}
        </View>
      )}
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
    width: '100%',
    height: 380,
    backgroundColor: '#fff',
    borderRadius: 20,
    padding: 16,
    elevation: 4,
    shadowColor: '#000',
    shadowOpacity: 0.12,
    shadowRadius: 8,
    shadowOffset: { width: 0, height: 3 },
  },
  face: { flex: 1, alignItems: 'center', justifyContent: 'center' },
  image: { width: '100%', flex: 1 },
  hint: { marginTop: 12, color: '#9ca3af', fontSize: 13 },
  word: { fontSize: 38, fontWeight: '700', color: '#111827', textAlign: 'center' },
  example: { marginTop: 16, fontSize: 16, color: '#6b7280', textAlign: 'center', fontStyle: 'italic' },
});
```

src/components/ReviewButtons.tsx

```
import React from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { ReviewQuality } from '../types/Card';

interface Props {
  onSelect: (quality: ReviewQuality) => void;
  disabled?: boolean;
}

const OPTIONS: { label: string; quality: ReviewQuality; color: string }[] = [
  { label: 'Otra vez', quality: 0, color: '#ef4444' },
  { label: 'Dificil', quality: 1, color: '#f59e0b' },
  { label: 'Bien', quality: 2, color: '#10b981' },
  { label: 'Facil', quality: 3, color: '#3b82f6' },
];

export default function ReviewButtons({ onSelect, disabled }: Props) {
  return (
    <View style={styles.row}>
      {OPTIONS.map((o) => (
        <Pressable
          key={o.quality}
          disabled={disabled}
          onPress={() => onSelect(o.quality)}
          style={[styles.button, { backgroundColor: o.color, opacity: disabled ? 0.5 : 1
        ]}]
        >
          <Text style={styles.text}>{o.label}</Text>
        </Pressable>
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', gap: 8, width: '100%' },
  button: { flex: 1, paddingVertical: 14, borderRadius: 12, alignItems: 'center' },
  text: { color: 'fff', fontWeight: '600', fontSize: 14 },
});
```

6.6 Pantallas

src/screens/HomeScreen.tsx

```
import React, { useCallback, useState } from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { useFocusEffect } from '@react-navigation/native';
import { getDueCards } from '../services/api';
import { useSession } from '../context/SessionContext';

export default function HomeScreen({ navigation }: any) {
  const { language } = useSession();
  const [dueCount, setDueCount] = useState<number | null>(null);
  const [error, setError] = useState<string | null>(null);

  useFocusEffect(
    useCallback(() => {
      setError(null);
      setDueCount(null);
      getDueCards(language, 100)
        .then((cards) => setDueCount(cards.length))
        .catch((e) => setError(e.message));
    }, [language])
  );

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Kian</Text>
      <Text style={styles.subtitle}>Idioma actual: {language.toUpperCase()}</Text>

      {error ? (
        <Text style={styles.error}>{error}</Text>
      ) : (
        <Text style={styles.count}>
          {dueCount === null ? '...' : dueCount} tarjetas por repasar
        </Text>
      )}

      <Pressable style={styles.button} onPress={() => navigation.navigate('Estudiar')}>
        <Text style={styles.buttonText}>Comenzar a estudiar</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center', justifyContent: 'center', padding: 24,
    backgroundColor: '#f9fafb' },
  title: { fontSize: 32, fontWeight: '700', color: '#111827' },
  subtitle: { marginTop: 8, fontSize: 16, color: '#6b7280' },
  count: { marginVertical: 32, fontSize: 20, color: '#4f46e5', fontWeight: '600' },
  error: { marginVertical: 32, color: '#ef4444', textAlign: 'center' },
  button: { backgroundColor: '#4f46e5', paddingVertical: 14, paddingHorizontal: 28,
    borderRadius: 12, marginTop: 16 },
  buttonText: { color: '#fff', fontWeight: '600', fontSize: 16 },
});
```

src/screens/StudyScreen.tsx

```
import React, { useCallback, useState } from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { useFocusEffect } from '@react-navigation/native';
import Flashcard from '../components/Flashcard';
import ReviewButtons from '../components/ReviewButtons';
import Loading from '../components/Loading';
import { getDueCards, sendReview } from '../services/api';
import { useSession } from '../context/SessionContext';
import { Card, ReviewQuality } from '../types/Card';

export default function StudyScreen() {
  const { language } = useSession();
  const [cards, setCards] = useState<Card[]>([]);
  const [index, setIndex] = useState(0);
  const [flipped, setFlipped] = useState(false);
  const [loading, setLoading] = useState(true);
  const [sending, setSending] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const load = useCallback(() => {
    setLoading(true);
    setError(null);
    getDueCards(language)
      .then((data) => {
        setCards(data);
        setIndex(0);
        setFlipped(false);
      })
      .catch((e) => setError(e.message))
      .finally(() => setLoading(false));
  }, [language]);

  useFocusEffect(load);

  const handleAnswer = async (quality: ReviewQuality) => {
    const card = cards[index];
    setSending(true);
    try {
      await sendReview(card.id, quality);
      setFlipped(false);
      setIndex((i) => i + 1);
    } catch (e: any) {
      setError(e.message);
    } finally {
      setSending(false);
    }
  };

  if (loading) return <Loading />;

  if (error) {
    return (
      <View style={styles.center}>
        <Text style={styles.error}>{error}</Text>
        <Pressable style={styles.button} onPress={load}>
          <Text style={styles.buttonText}>Reintentar</Text>
        </Pressable>
      </View>
    );
  }

  return (
    <Flashcard
      card={cards[index]}
      flipped={flipped}
      loading={loading}
      sending={sending}
      error={error}
      handleAnswer={handleAnswer}
    />
  );
}
```

```

    </View>
  );
}

if (index >= cards.length) {
  return (
    <View style={styles.center}>
      <Text style={styles.done}>Listo por ahora!</Text>
      <Text style={styles.message}>
        No tienes mas tarjetas pendientes en {language.toUpperCase()}.
      </Text>
      <Pressable style={styles.button} onPress={load}>
        <Text style={styles.buttonText}>Buscar mas</Text>
      </Pressable>
    </View>
  );
}

return (
  <View style={styles.container}>
    <Text style={styles.progress}>
      {index + 1} / {cards.length}
    </Text>

    <Flashcard card={cards[index]} flipped={flipped} onFlip={() => setFlipped((f) => !f)}
  />

    <View style={styles.actions}>
      {flipped ? (
        <ReviewButtons onSelect={handleAnswer} disabled={sending} />
      ) : (
        <Pressable style={styles.button} onPress={() => setFlipped(true)}>
          <Text style={styles.buttonText}>Mostrar respuesta</Text>
        </Pressable>
      )}
    </View>
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#f9fafb', alignItems: 'center' },
  center: { flex: 1, alignItems: 'center', justifyContent: 'center', padding: 24,
    backgroundColor: '#f9fafb' },
  progress: { marginBottom: 12, color: '#6b7280', fontSize: 14 },
  actions: { marginTop: 24, width: '100%', alignItems: 'center' },
  message: { fontSize: 16, color: '#6b7280', textAlign: 'center', marginVertical: 12 },
  done: { fontSize: 28, fontWeight: '700', color: '#111827' },
  error: { color: '#ef4444', textAlign: 'center', marginBottom: 16 },
  button: { backgroundColor: '#4f46e5', paddingVertical: 14, paddingHorizontal: 28,
    borderRadius: 12 },
  buttonText: { color: '#fff', fontWeight: '600', fontSize: 16 },
});

```

src/screens/LanguagesScreen.tsx

```
import React from 'react';
import { FlatList, Pressable, StyleSheet, Text, View } from 'react-native';
import { useSession } from '../context/SessionContext';

const LANGUAGES = [
  { code: 'en', name: 'Ingles' },
  { code: 'es', name: 'Espanol' },
  { code: 'fr', name: 'Frances' },
  { code: 'de', name: 'Aleman' },
  { code: 'it', name: 'Italiano' },
  { code: 'pt', name: 'Portugues' },
  { code: 'ja', name: 'Japones' },
];

export default function LanguagesScreen() {
  const { language, setLanguage } = useSession();

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Idioma de aprendizaje</Text>
      <FlatList
        data={LANGUAGES}
        keyExtractor={(item) => item.code}
        renderItem={({ item }) => {
          const selected = item.code === language;
          return (
            <Pressable
              style={[styles.item, selected && styles.itemSelected]}
              onPress={() => setLanguage(item.code)}
            >
              <Text style={[styles.itemText, selected && styles.itemTextSelected]}>
                {item.name} ({item.code})
              </Text>
              {selected && <Text style={styles.check}>OK</Text>}
            </Pressable>
          );
        }}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#f9fafb' },
  title: { fontSize: 22, fontWeight: '700', marginBottom: 16, color: '#111827' },
  item: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    padding: 16,
    backgroundColor: '#fff',
    borderRadius: 12,
    marginBottom: 10,
    borderWidth: 1,
    borderColor: '#e5e7eb',
  },
  itemSelected: { borderColor: '#4f46e5', backgroundColor: '#eef2ff' },
  itemText: { fontSize: 16, color: '#374151' },
});
```

```
    itemTextSelected: { color: '#4f46e5', fontWeight: '600' },  
    check: { color: '#4f46e5', fontSize: 18, fontWeight: '700' },  
  });
```

6.7 Navegación

src/navigation/AppNavigation.tsx

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import HomeScreen from '../screens/HomeScreen';
import StudyScreen from '../screens/StudyScreen';
import LanguagesScreen from '../screens/LanguagesScreen';

const Tab = createBottomTabNavigator();

export default function AppNavigation() {
  return (
    <NavigationContainer>
      <Tab.Navigator screenOptions={{ tabBarActiveTintColor: '#4f46e5' }}>
        <Tab.Screen name="Inicio" component={HomeScreen} />
        <Tab.Screen name="Estudiar" component={StudyScreen} />
        <Tab.Screen name="Idiomas" component={LanguagesScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

6.8 App.tsx (raíz)

```
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import AppNavigation from '../src/navigation/AppNavigation';
import { SessionProvider, useSession } from '../src/context/SessionContext';
import Loading from '../src/components/Loading';

function Root() {
  const { loaded } = useSession();
  return loaded ? <AppNavigation /> : <Loading />;
}

export default function App() {
  return (
    <SessionProvider>
      <StatusBar style="auto" />
      <Root />
    </SessionProvider>
  );
}
```

7. Cómo correr el proyecto

7.1 Backend

```
cd KIAN
dotnet run --launch-profile http
```

Revisar en la consola la línea “Now listening on: ...” para confirmar el puerto. Probar en el navegador: `http://localhost:[PUERTO]/swagger` y ejecutar `GET /api/cards` (debe responder 200 con [] si no hay tarjetas aún).

7.2 Frontend

```
cd kianMobile
npm install
npx expo start
```

- Presionar “a” → abre en emulador Android
- Presionar “i” → abre en simulador iOS (solo Mac)
- Escanear el código QR con la app Expo Go → celular físico

7.3 Configuración de red según el entorno de prueba

Escenario	HOST en api.ts	Backend debe escuchar en
Emulador Android (AVD)	10.0.2.2	<code>http://0.0.0.0:[PUERTO]</code>
Emulador Genymotion	10.0.3.2	<code>http://0.0.0.0:[PUERTO]</code>
Simulador iOS (Mac)	localhost	<code>http://localhost:[PUERTO]</code>
Celular físico (Expo Go)	IP LAN de la PC ([IP-DE-TU-PC])	<code>http://0.0.0.0:[PUERTO]</code> , misma red WiFi

Alternativa robusta para emulador Android si 10.0.2.2 falla: usar “`adb reverse tcp:[PUERTO] tcp:[PUERTO]`” y dejar `HOST = 'localhost'` en `api.ts`.

8. Flujo de datos de una tarjeta

1. El usuario abre la app y selecciona un idioma en la pantalla Idiomas.
2. La app llama a GET /api/reviews/next?language=xx.
3. El backend consulta CardReviews en Azure SQL filtrando por NextReviewAt <= ahora y el idioma.
4. Por cada tarjeta, se genera una URL temporal (SAS) hacia Blob Storage.
5. La app muestra la imagen; el usuario la voltea y ve la traducción.
6. El usuario califica (Otra vez / Difícil / Bien / Fácil) mediante POST /api/reviews.
7. SpacedRepetitionService recalcula IntervalDays, EaseFactor, Repetitions, Lapses y NextReviewAt.
8. Se actualiza CardReviews y se muestra la siguiente tarjeta.

9. Endpoints disponibles

Método	Endpoint	Propósito
GET	/api/cards	Todas las tarjetas del usuario
GET	/api/cards?language=en	Tarjetas filtradas por idioma
GET	/api/cards/{id}	Detalle de una tarjeta
POST	/api/cards	Crear tarjeta (multipart/form-data: Image, LanguageCode, TranslatedText, ExampleSentence)
POST	/api/cards/{id}/translations	Agregar traducción a tarjeta existente
GET	/api/reviews/next?language=en&limit=20	Tarjetas pendientes de repaso
POST	/api/reviews	Registrar respuesta ({ cardId, quality }, quality 0–3)

10. Checklist de configuración de secretos

```
dotnet user-secrets list
```

Debe mostrar:

- ConnectionStrings:DefaultConnection
- AzureStorage:ContainerName (opcional, ya en appsettings.json)
- AzureStorage:ConnectionString

11. .gitignore recomendado

Carpeta raíz: *[NOMBRE DEL REPOSITORIO]/*

```
# Backend
KIAN/bin/
KIAN/obj/
.vs/

# Frontend
kianMobile/node_modules/
kianMobile/.expo/
```

12. Tabla de errores comunes

Error	Causa	Solución
Client with IP address ... is not allowed	Falta regla de firewall en Azure SQL	Agregar tu IP en Networking → Firewall rules
Login failed for user	Usuario/contraseña incorrectos	Revisar User Secrets
Cannot open database ... requested by the login	Nombre de base mal escrito	Revisar Initial Catalog
Falta AzureStorage:ConnectionString	No configurado	Repetir dotnet user-secrets set de storage
Invalid object name 'Users'	Tablas no creadas en esa base	Ejecutar el script SQL
fetch failed: ConnectException ... 10.0.2.2	Backend no escucha en 0.0.0.0 o firewall de Windows bloquea	Editar applicationUrl en launchSettings.json, permitir el puerto en firewall, o usar adb reverse
Unable to resolve react-native-web/...	Se probó la opción “web” sin tener esas dependencias	No usar “web”; probar con Android/iOS/Expo Go
App muestra “Open up App.tsx to start working”	No se reemplazó el App.tsx de plantilla	Sobrescribir con el código de la sección 6.8

Documento generado para el proyecto KIAN. Completar los campos entre corchetes [] antes de entregar o compartir este documento.